CS330 Operating Systems and Lab.

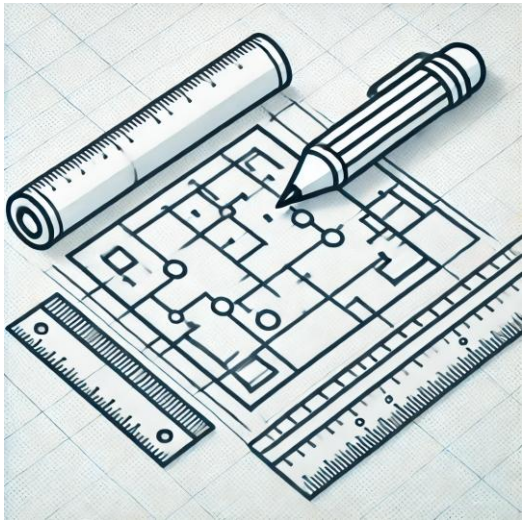
Computer System Organization

Spring 2026

Instructor : Insik Shin

How to Design OS ?

- A central theme in operating system design is how to organize the operating system
  - what is an operating system?
    - a good understanding of computer system structure can help
  - basic design problems as examples
    - identify system-level functionalities and break down them into component-level
    - design communication between key components



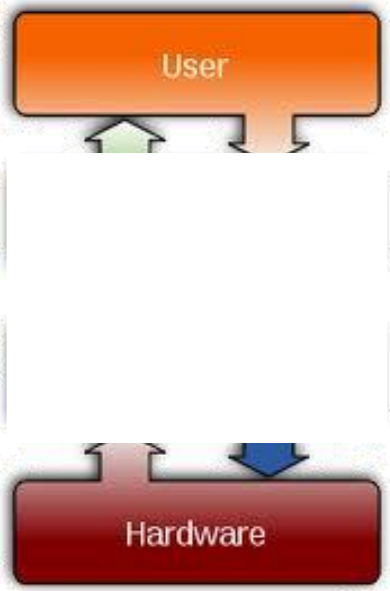
- Design: a blueprint that outlines the structure and functionality of a system before it is built (where a system is a collection of components)

How do We Tame Complexity?

Side Slide

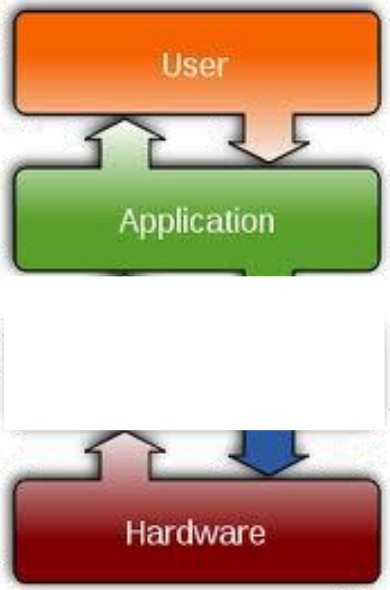
- Computer systems have diverse hardware components:
  - Different CPUs
    - Pentium, ARM, PowerPC, ColdFire
  - Various memory capacities and storage configurations
  - Different types of I/O devices
    - Mice, keyboards, sensors, cameras, touch screen
  - Different networking environments
    - Wired, Wireless (WiFi, 5G, Bluetooth), …
- Questions:
  - Do all programs need to be modified for different hardware?
  - Should each programmer write their own version of common system functionalities?
  - Can a faulty program crash the entire system?
  - Should every program have unrestricted access to all hardware?

Computer System Structure



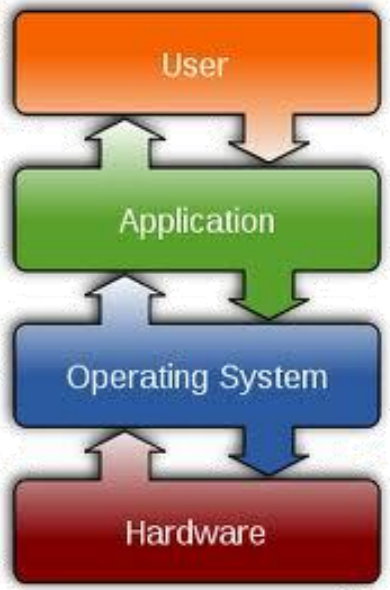
- Computer system can fall into four compone
  - Users
    - People, machines, other computers
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices

Computer System Structure



- Computer system can fall into four compone
  - Users
    - People, machines, other computers
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Application programs – define the ways in which the system resources are used to support users' demands
    - Word processors, web browsers, database systems, video games

Computer System Structure



- Computer system can fall into four components
  - Users
    - People, machines, other computers
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Application programs – define the ways in which the system resources are used to support users' demands
    - Word processors, web browsers, database systems, video games
  - Operating system - controls and coordinates use of hardware among various applications and users
    - A program that acts as an intermediary between a user of a computer and the computer hardware

What Operating Systems Do

- Silberschatz and Galvin: “An OS is similar to a government”

- OS as a Facilitator (“useful” abstractions):
  - Provides facilities/services that everyone needs
  - Standard libraries, windowing systems
  - Makes application programming easier, faster, less error-prone
- OS as a Traffic Cop:
  - Manages all resources
  - Settles conflicting requests for resources
  - Prevents errors and improper use of the computer

- Some features reflect both tasks:
  - File system is needed by everyone (Facilitator) …
  - … but File system must be protected (Traffic Cop)

Operating System Definition

- No universally accepted definition
- "*Everything a vendor ships when you order an operating system*" is good approximation
  - But varies wildly

Browser War

CNN Money

Netscape sues Microsoft

January 22, 2002: 5:53 p.m. ET

Federal antitrust case seeks damages for Microsoft's "anticompetitive conduct."

NEW YORK (CNN/Money) - Netscape Communications filed an antitrust lawsuit against Microsoft Corp. Tuesday afternoon, citing the same grounds that were used in the U.S. Justice Department's suit against the software provider.

Netscape, a unit of AOL Time Warner Inc. (AOL: Research, Estimates), filed a seven-count suit in U.S. District Court for the District of Columbia, charging Microsoft (MSFT: Research, Estimates) harmed Netscape in a series of illegal acts aimed at promoting Microsoft's Internet Explorer browser at the expense of Netscape Navigator. CNN/Money also is a unit of AOL Time Warner.

SAVE THIS
EMAIL THIS
PRINT THIS
MOST POPULAR

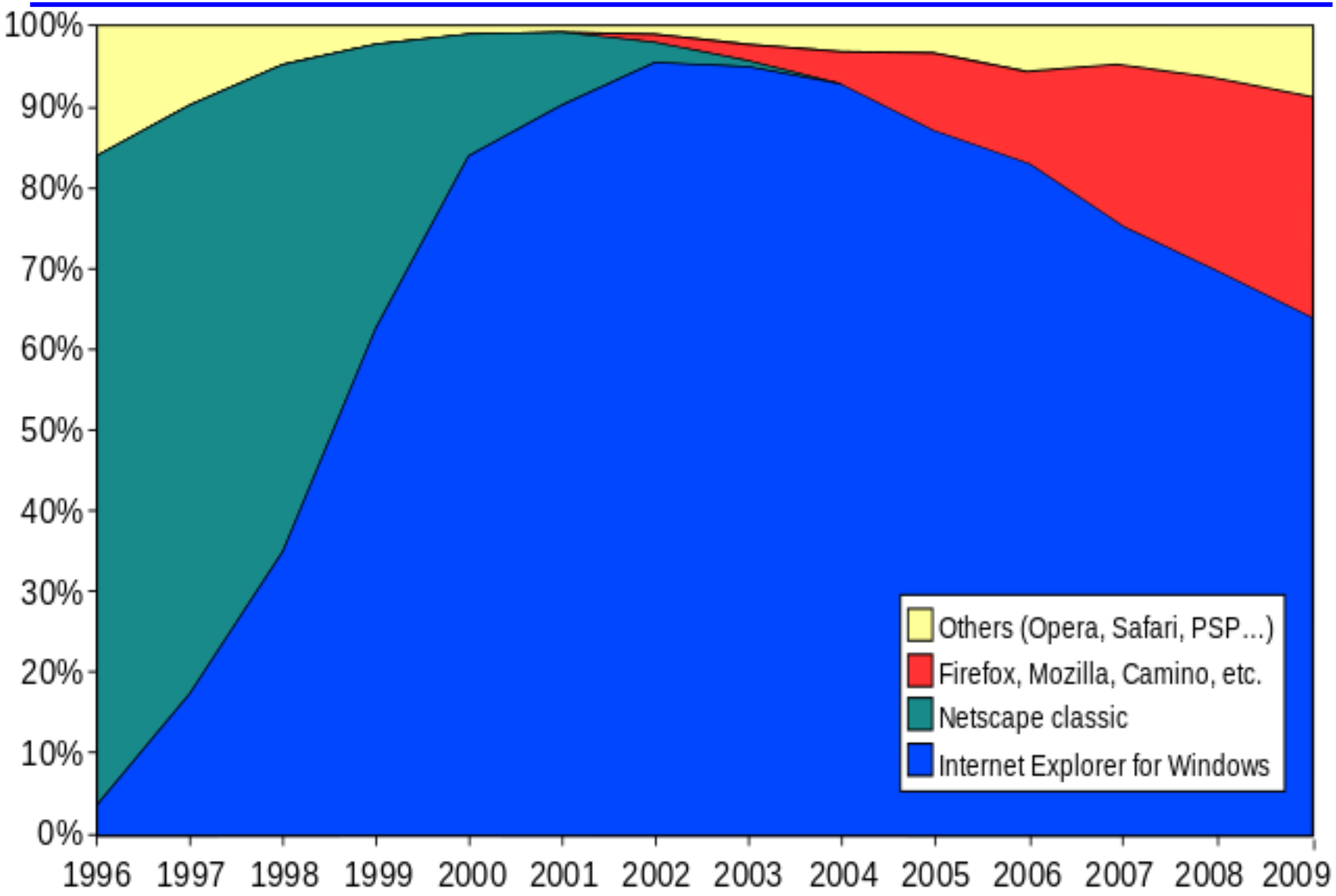
RELATED STORIES

- **MSFT private settlement denied** - Jan. 11, 2002
- **Justice Department and Microsoft settle** - Nov. 2, 2001

RELATED LINKS

- **Microsoft Corp.**

Browser Wars



Operating System Definition

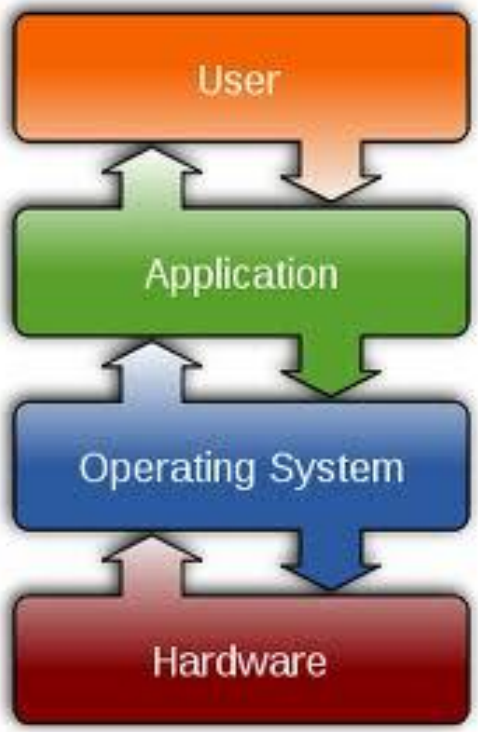
- No universally accepted definition
- "*Everything a vendor ships when you order an operating system*" is good approximation
  - But varies wildly
- "*The one program running at all times on the computer*" is the kernel
  - Everything else is either a system program (ships with the operating system) or an application program

Computer System Structure

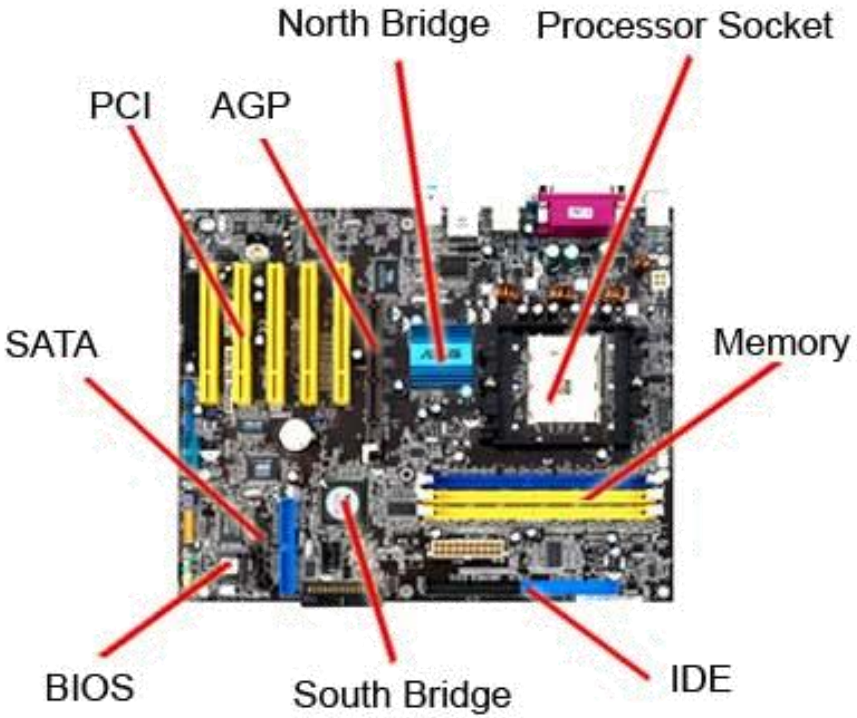
- Computer system can be divided into four components

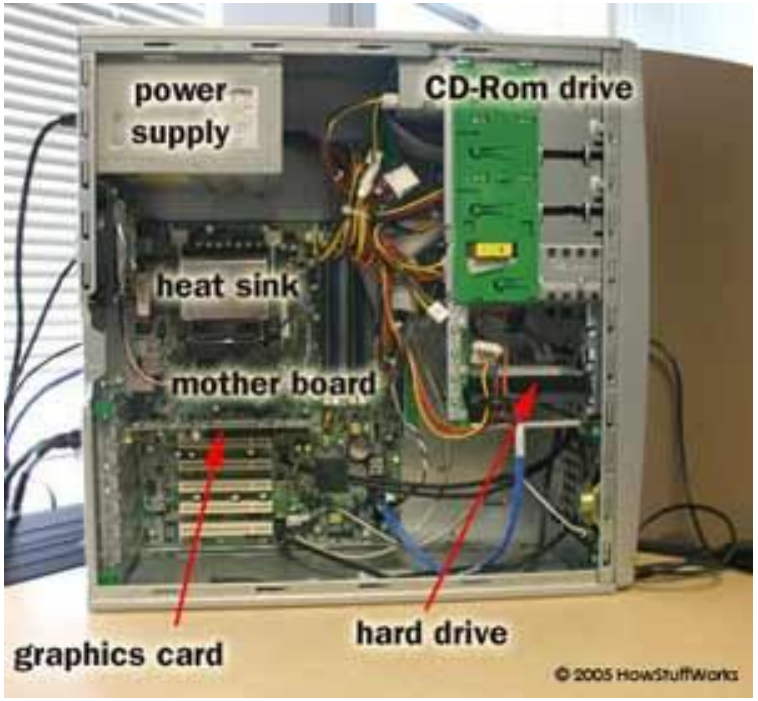
How to design interactions between components?

- Hardware – provides basic computing resources
  - CPU, memory, I/
- Application progr in which the system resources are usec ıg problems of the users
  - Word processors, tems, video games
- Operating system ıates use of hardware among various apj



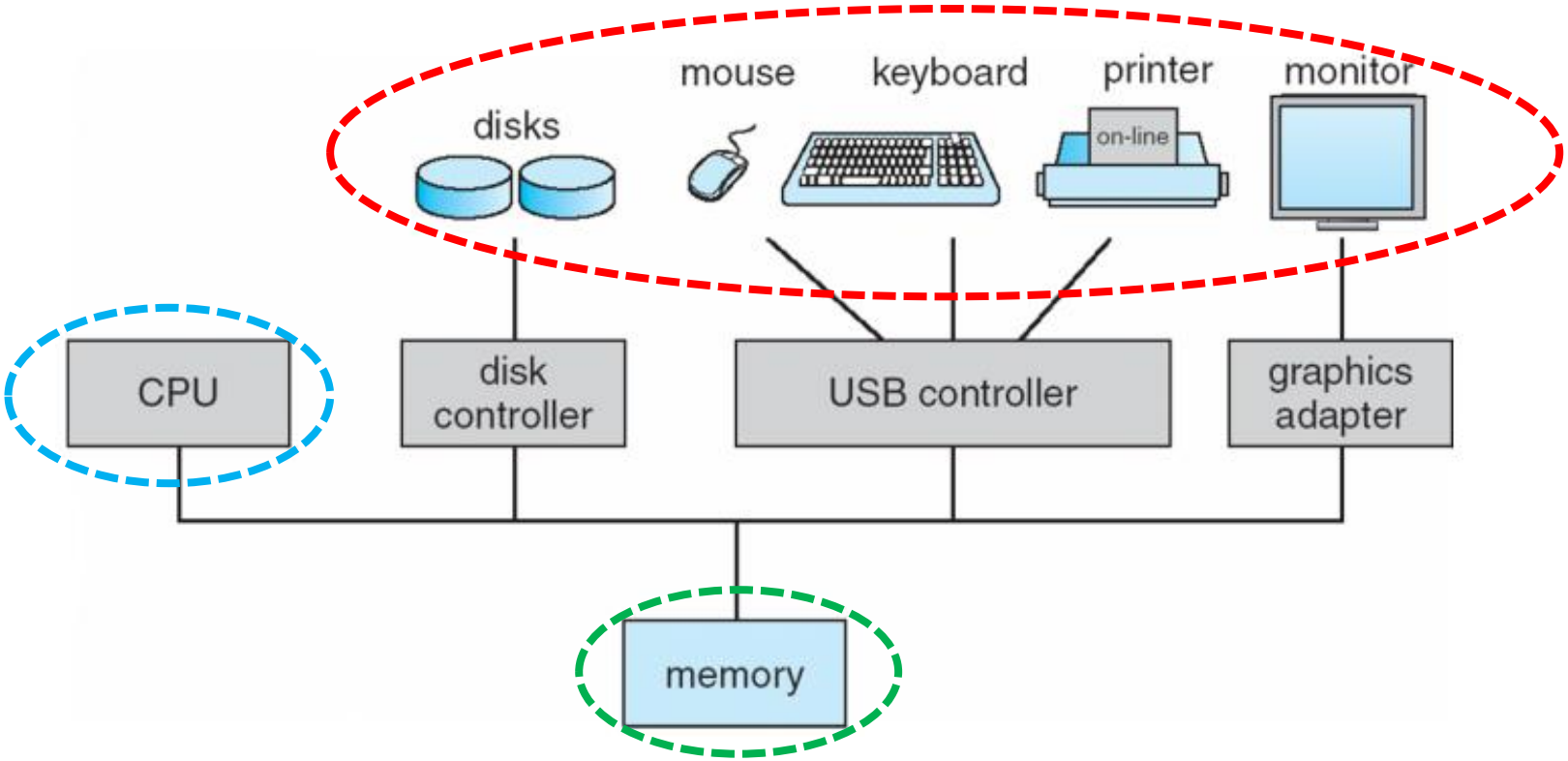
Computer System Organization





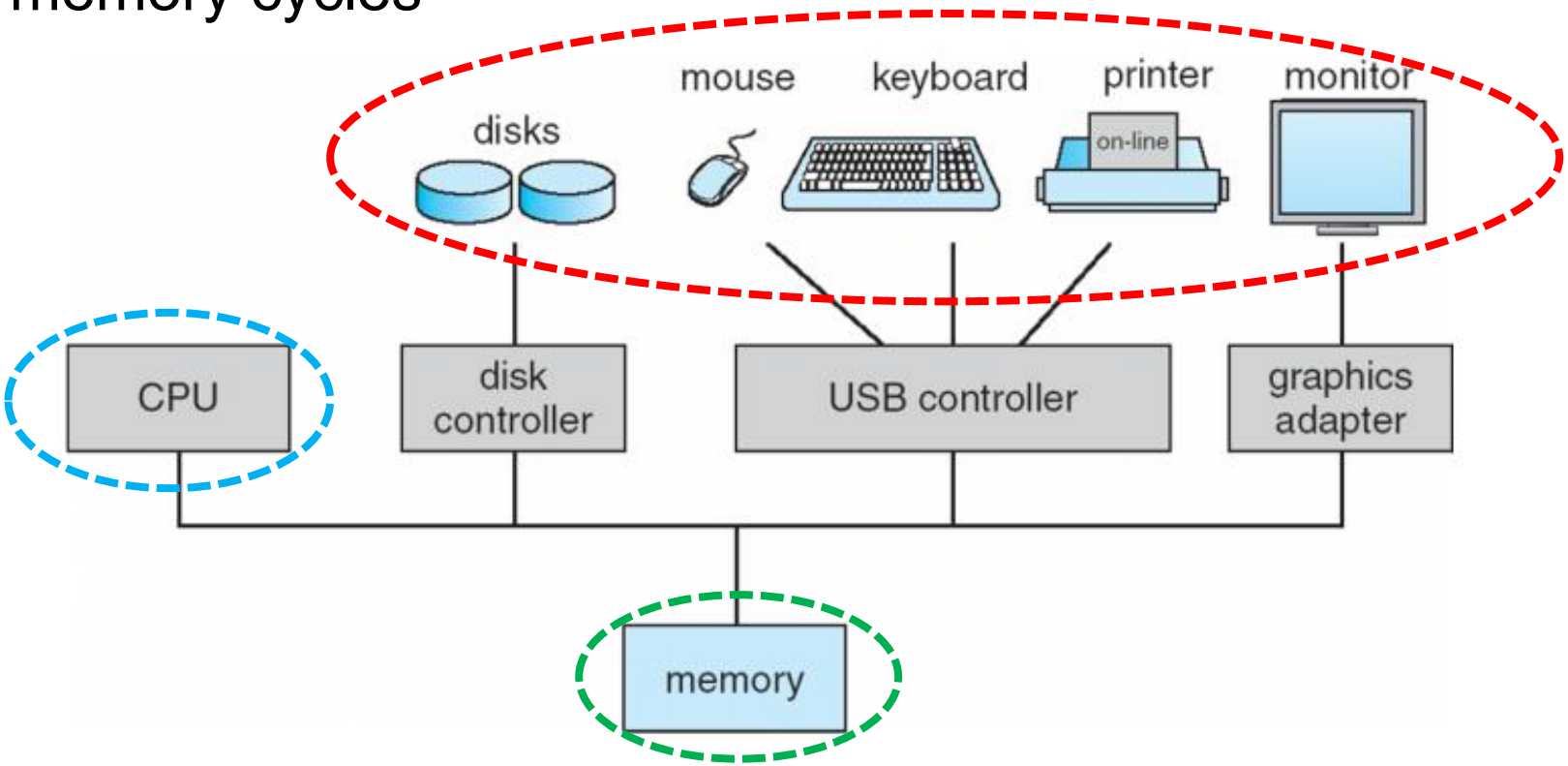
Computer System Organization

- A computer system consist of:
  - CPUs: execution instructions
  - Device controllers: manage I/O devices
  - Memory: communication medium between components



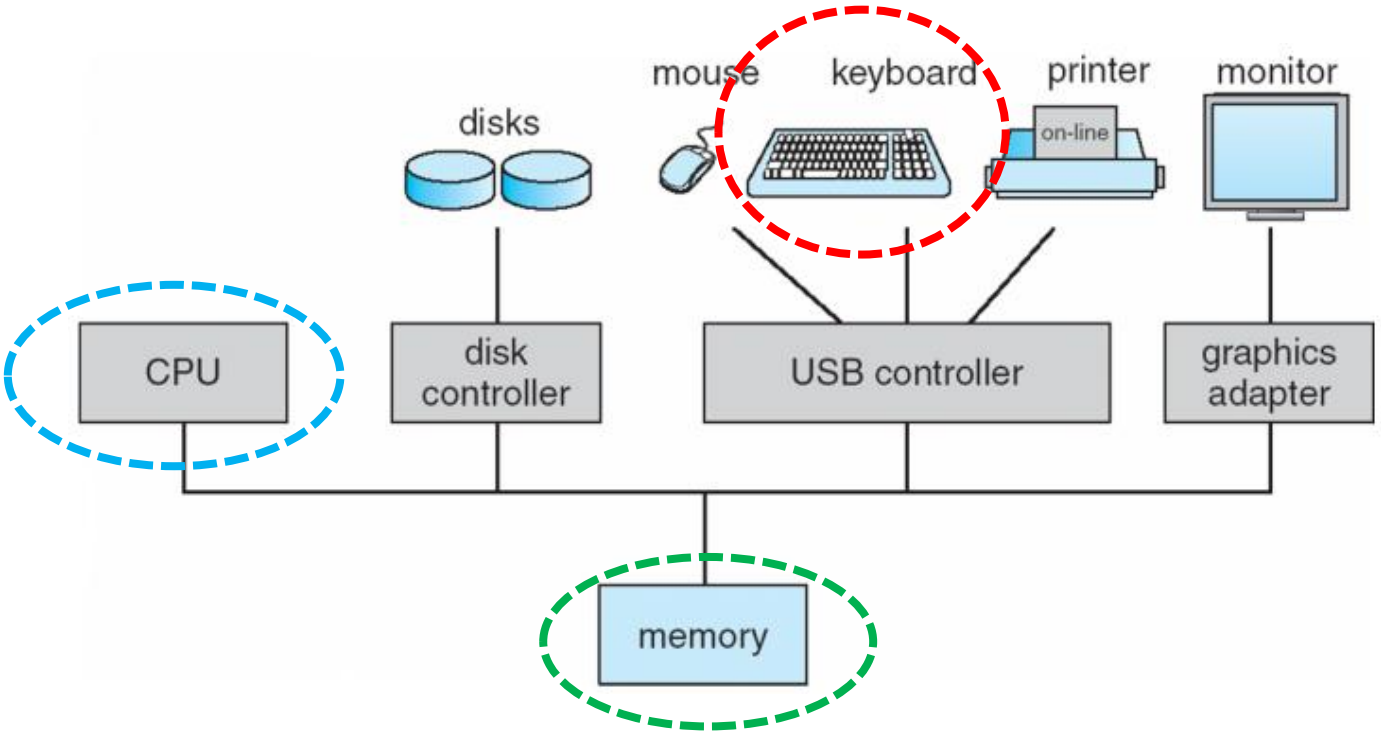
Computer System Organization

- One or more CPUs, device controllers connect through common bus providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles



Computer-System Operation

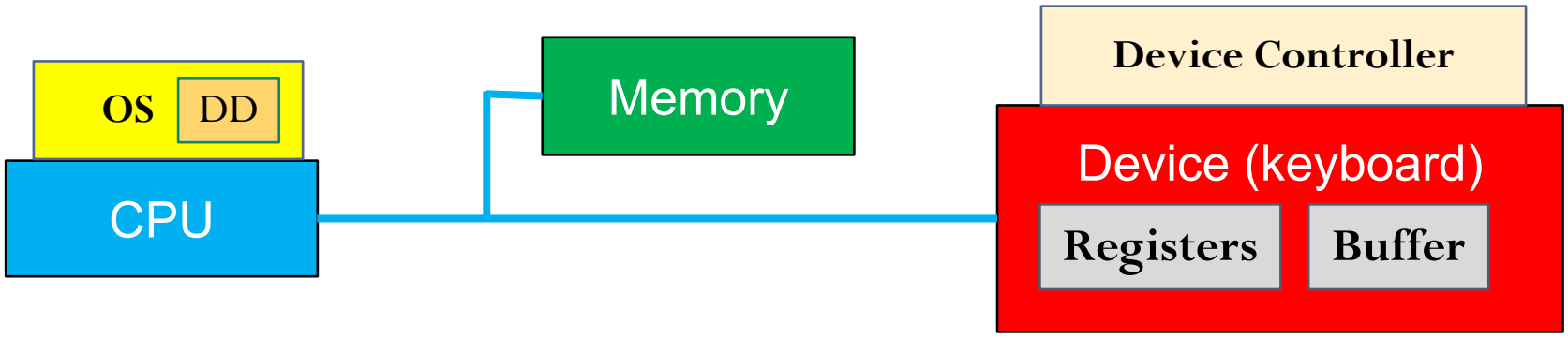
- I/O devices and the CPU can execute concurrently
- How do they communicate ?
  - Each device controller is in charge of a particular device type
  - Each device controller has a local buffer
  - CPU moves data from/to main memory to/from local buffers



Computer-System Operation

- Each I/O device has its own device controller (DC), and OS has a device driver (DD) for each controller
- How do they communicate ?
  1. DD loads proper registers within DC (i.e., command)
  2. DC looks at the registers and determines what to do (i.e., read a character from the keyboard)
  3. DC moves data from the device to its local buffer
  4. DD moves data from the local buffer to main memory

- Any potential issues and solutions?



Communication over Events

- Interrupt
  - A signal to CPU emitted by HW and SW indicating an event that needs immediate attention
    - HW sends a signal to the CPU, thru the system bus
    - SW executes a special operation called a *system call*

The 6502's interrupt latency is one of the 8-bit world's fastest.

Interrupt-driven I/O Operation

- I/O is an important part of OS, since it is critical to reliability and performance
- Each I/O device has its own device controller (DC), and OS has a device driver (DD) for each controller
- Interrupt-driven I/O
  1. DD loads proper registers within DC (i.e., command)
  2. DC looks at the registers and determines what to do (i.e., read a character from the keyboard)
  3. DC moves data from the device to its local buffer
  4. Once completion, DC informs DD via an interrupt
  5. DD returns the control to the OS, possibly returning the data (or status)

Communication over Events

- Interrupt
  - Hardware and software have their own events and signal the occurrence of the events to the CPU by an ***interrupt***
    - H/W sends a signal to the CPU, thru the system bus
    - S/W executes a special operation called a *system call*
- CPU, when interrupted, transfers control to interrupt service routines (ISR), and resumes its interrupted computation when ISR is done

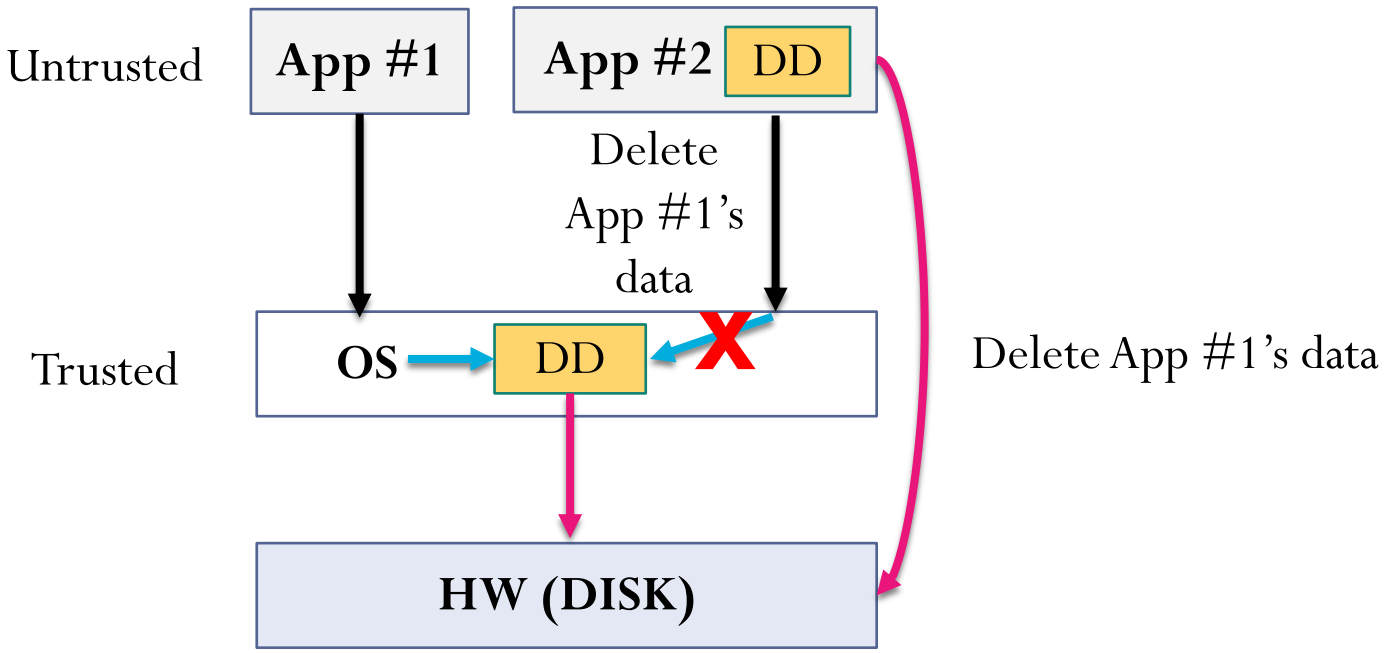
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A ***trap*** (or an *exception*) is a software-generated interrupt caused either by an error (i.e., divide-by-zero or invalid memory access) or a user request
- An operating system is **interrupt driven**

Operating-System Operations

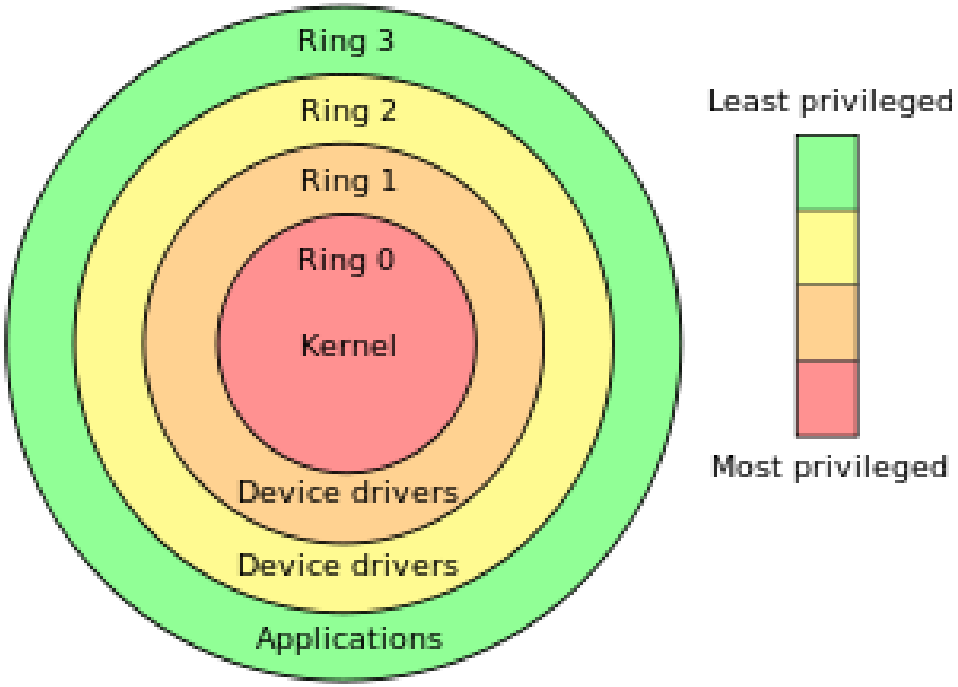
- OS and users share HW and SW resources
- Users could cause problems, such as infinite loop or modifying other user programs or OS
- Needs to protect OS and users from erroneous (or malicious) user programs
  - Example: what if some applications try to delete a file ?

- Example: what if some applications try to delete a file ?



Protection Mechanisms

- Protection mechanisms
  - Dual mode operation (or ring mode)
    - mode bit is provided by hardware
  - Privileged I/O instructions
  - Memory protection mechanism (later in this semester)

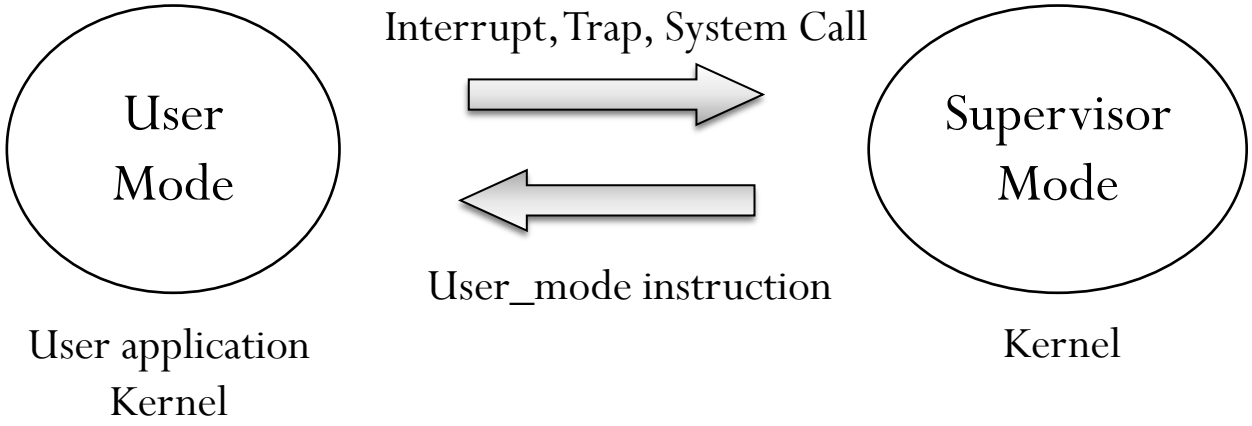


Dual Mode Operation

- CPU operates in two or more states:
  - Supervisor (kernel) mode
    - All instructions may be executed
    - This is a *privileged* mode
    - Only parts of the OS run in this mode
    - User threads *never* run directly in this mode
  - User mode
    - Some instructions are “privileged”
    - Attempt to execute a privileged instruction results in a trap
    - User threads execute in this mode
- Examples of privileged instructions
  - I/O instructions, halt, reset, mask interrupts, set interval timer, modify page table registers

Life Cycle

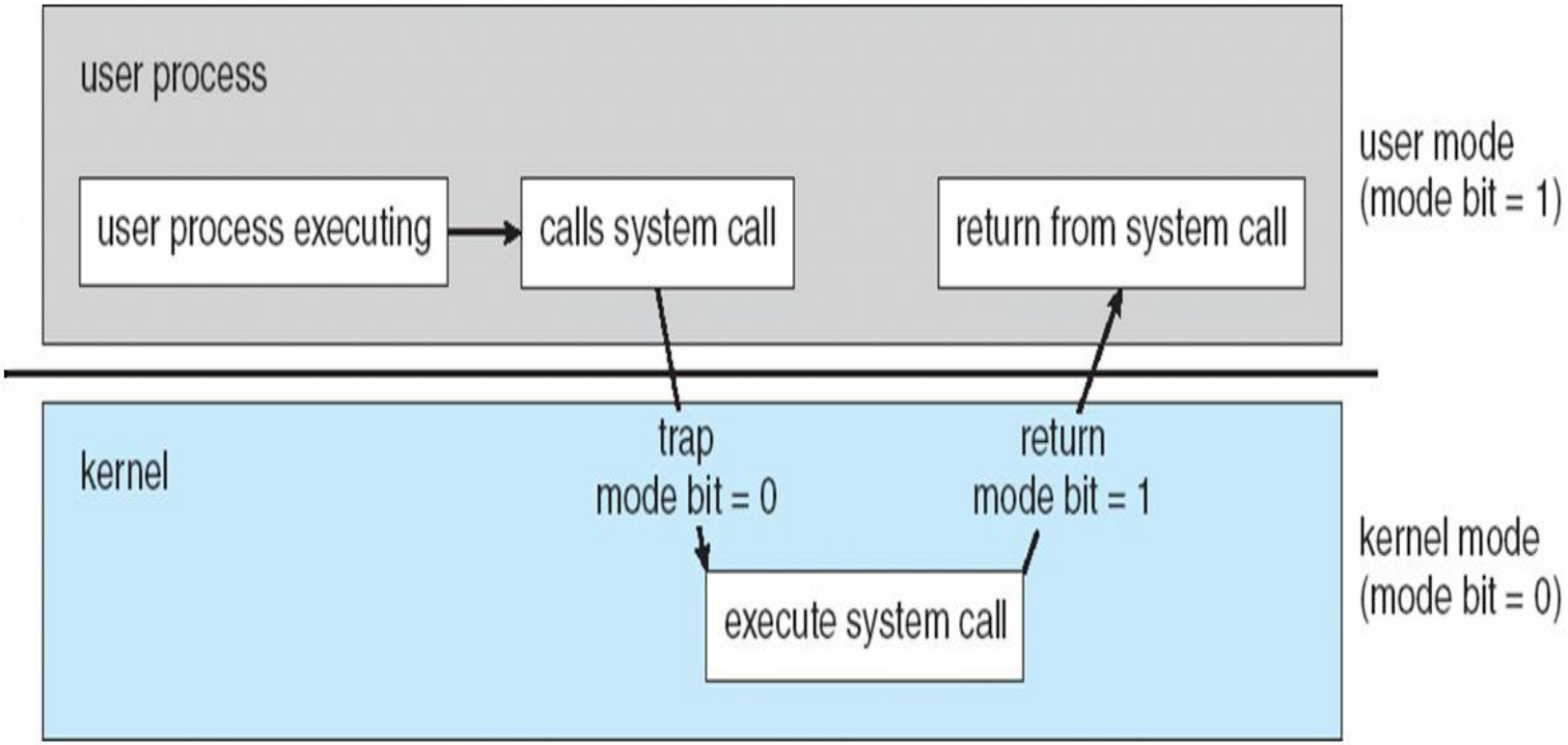
- Schema of system operation
  - System powered on; in supervisor mode
  - System boots, kernel initializes, still in supervisor mode
  - System enters user mode to run user programs
- Should be there a user mode instruction to enter supervisor mode ?
- How can supervisor mode be re-entered?



- Differences, similarities btw. interrupts, traps, system calls?

Transition from User to Kernel Mode

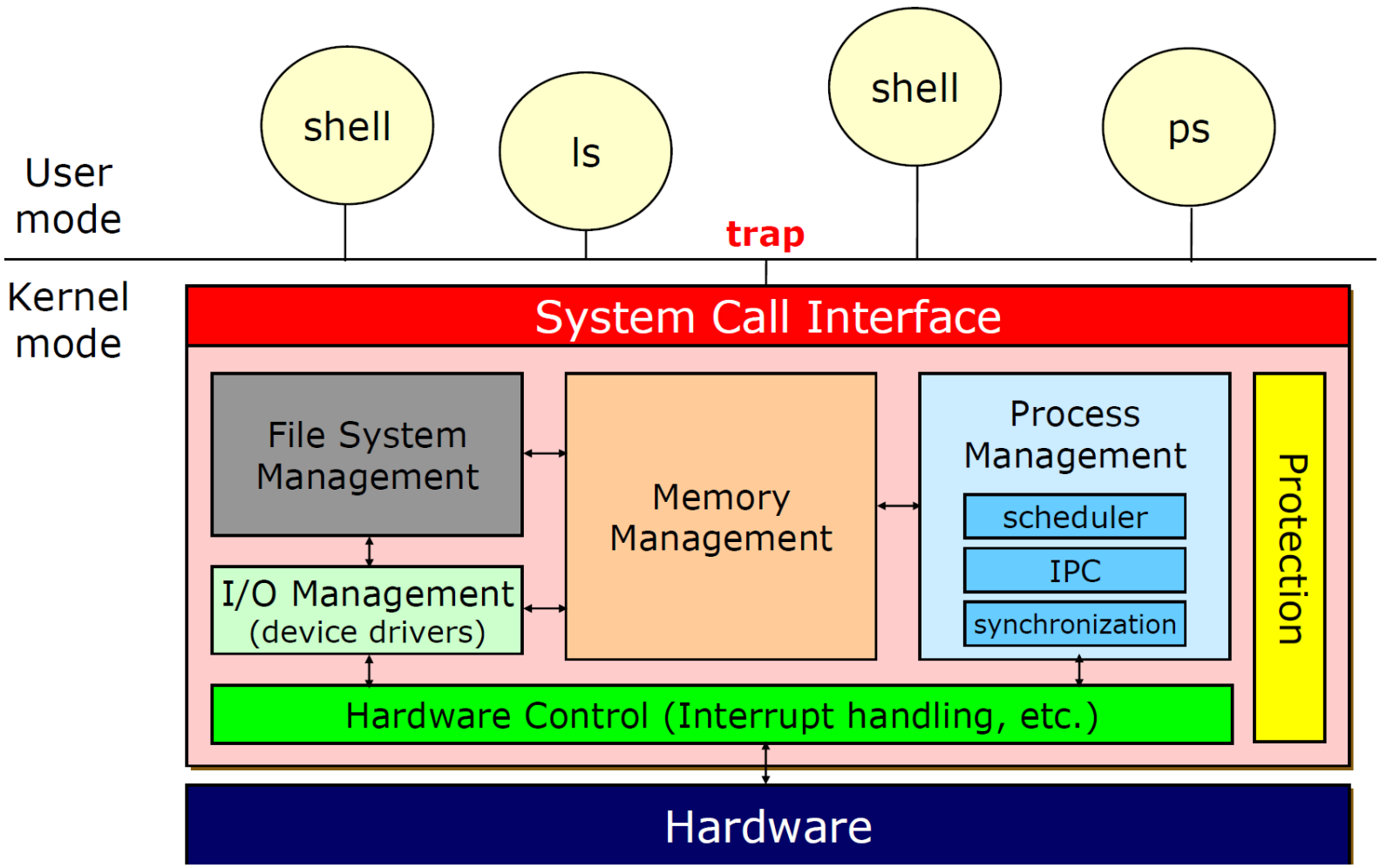
- Transition through systems calls



Operating System Services

- OS provides useful services to users
  - User interface
  - Program execution
  - I/O operations
  - File-system manipulation
  - Communications
  - Error detection

Operating System Structure

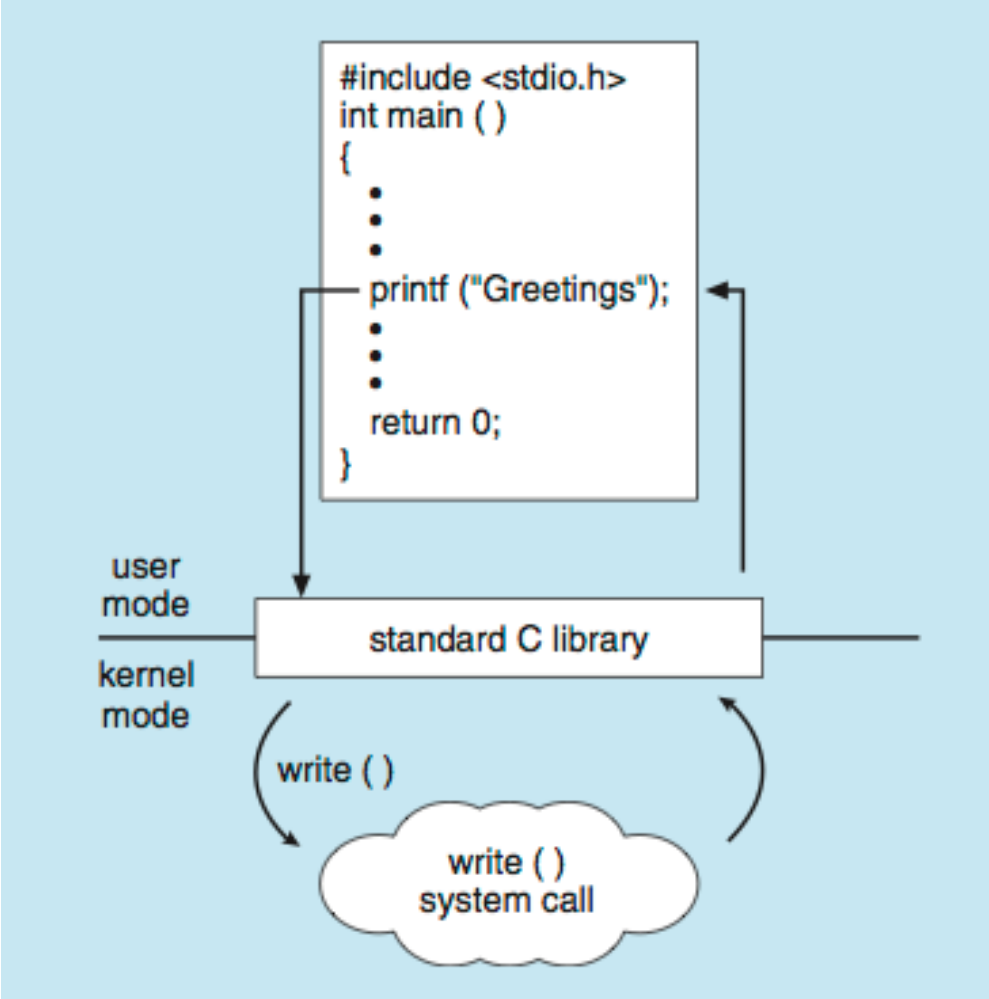


System Calls

- Programming interface to the services provided by the OS
- Mostly accessed by programs via a high-level Application Program Interface (API) rather than direct system call use
  - Three most common APIs:
    - Win32 API for Windows,
    - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and
    - Java API for the Java virtual machine (JVM)
- Why use APIs rather than system calls?

Standard C Library Example

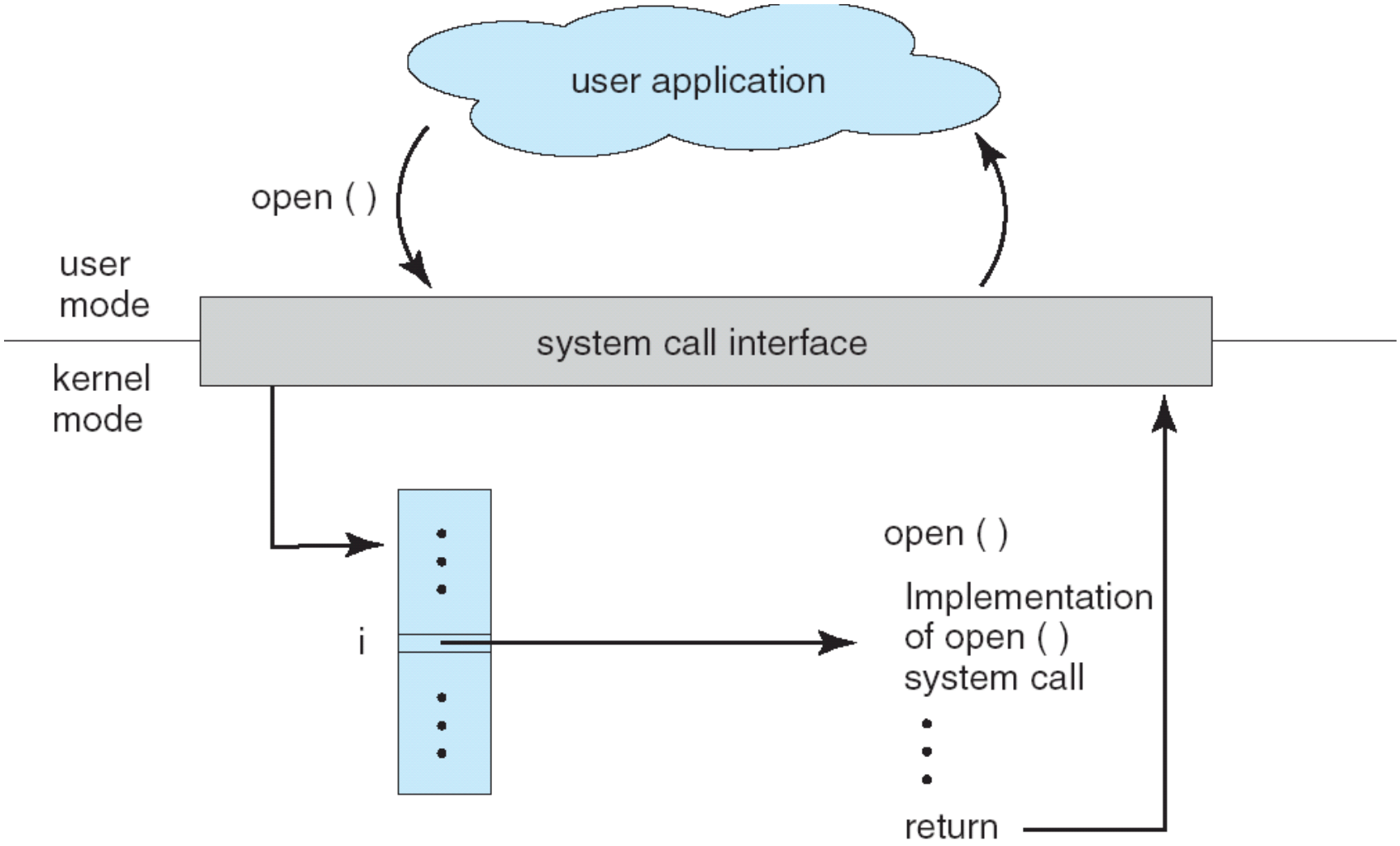
- C program invoking printf() library call, which calls write() system call



System Call Implementation

- Typically, a number associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller needs to know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship



System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call

- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in *registers*
    - In some cases, may be more parameters than registers
  - Parameters stored in a *block,* or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or *pushed,* onto the *stack* by the program and *popped* off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table